Vision-Language-Action Models

Machine Learning and Deep Learning – Presentation Notes

Assigned subtopics: RT-2, OpenVLA, Octo, π_0, Generalist Robot Policies, Vision-Language-Conditioned Control

1 The Big Picture: What a VLA Model Is

A Vision-Language-Action (VLA) model is a single neural network that takes **(camera image, natural-language instruction)** as input and outputs a **robot action** directly – with no separate, hand-engineered perception, planning, or control modules in between.

The core idea: take a model already pretrained on huge internet-scale image+text data (a Vision-Language Model – the kind behind image captioning and visual question answering), and fine-tune it so that instead of outputting text, it outputs robot actions. The underlying bet is that the semantic and world knowledge baked into the VLM (what a "banana" looks like, what "put X in Y" means, spatial relationships, object affordances) transfers to robot control and generalizes far better than training a robot policy from scratch on a small set of demonstrations.

This addresses a longstanding robotics problem: classic robot policies only work for the exact tasks and objects they were trained on. VLA models aim to be **generalist** – one set of weights that follows arbitrary language instructions across many tasks, objects, and sometimes even different robot embodiments.

2 How the Six Subtopics Relate to Each Other

These are not six independent topics – they form one lineage, framed by two organizing concepts.

"Generalist robot policies" — the *goal / category* all of these belong to
"Vision-language-conditioned control" — the *mechanism* that makes it possible

RT-2	OpenVLA	Octo	π_0
(closed, Google)	(open, 7B, OXE data)	(open, small, OXE data)	(open weights, flow matching)

- **RT-2** established the paradigm (2023): fine-tune a VLM to speak "action tokens" instead of words.
- **OpenVLA** and **Octo** are both open-source responses trained on the same large public dataset (**Open X-Embodiment**, roughly 1M robot trajectories pooled across ∼20 labs and robot types), but they diverge architecturally in *how* they turn language and vision into actions.
- π_0 is a further evolution – instead of discretizing actions into tokens (like RT-2/OpenVLA), it generates smooth, continuous, high-frequency actions via flow matching (related to diffusion models).
- **Generalist robot policies** is the umbrella term for what all four of these *are* – a shift away from one-task-one-policy toward one-model-many-tasks.
- **Vision-language-conditioned control** is *how* they work internally – the mechanism (attention over image patches and language tokens, jointly conditioning the action decoder) that each model implements differently.

A presentation covering all six bullet points naturally uses the last two as the framing/comparison narrative around the four concrete models.

3 Each Subtopic in Detail

3.1 RT-2 (Robotic Transformer 2)

Google DeepMind, 2023. Takes a pretrained VLM (PaLI-X or PaLM-E) and fine-tunes it *jointly* on web-scale vision-language data and robot trajectory data. Robot actions (7 numbers: x, y, z, roll, pitch, yaw, gripper) are discretized into bins and represented as extra "words" in the model's vocabulary, so the model literally outputs a string of action-tokens the same way it would output a text answer. Because it is co-trained on web data, it can follow instructions involving objects/concepts never seen in robot demonstrations (e.g. "pick up the extinct animal" $\rightarrow$ picks the dinosaur toy), showing genuine semantic transfer. **Not open-sourced** – no public weights.

3.2 OpenVLA

Stanford / Berkeley / Toyota Research / Google, 2024. An open-source response to RT-2. 7B parameters, built on a Llama-2 backbone fused with visual encoders (DINOv2 + SigLIP features). Trained on 970k trajectories from Open X-Embodiment. Same discrete-action-token approach as RT-2 (autoregressively predicts 256-bin-discretized action dimensions). Ships with reference inference code: feed one RGB image plus a text instruction, and it returns a 7-DoF action vector for one timestep. This is the standard reference point for "what a VLA model actually is" in current coursework.

3.3 Octo

UC Berkeley, 2024. Also trained on Open X-Embodiment, but architecturally different: a modular transformer where observations (multiple camera views, proprioception, goal images, or language) are tokenized flexibly and combined via cross-attention, and the action head is a **diffusion policy** that predicts a *chunk* of future actions at once, rather than one action at a time autoregressively. Much smaller than OpenVLA – 27M ("Octo-Small") or 93M ("Octo-Base") parameters, vs. 7B – designed explicitly to be fast to fine-tune on a new robot with limited demonstrations. This is also the practical reason it is the easiest of the four to actually run live.

3.4 π_0 (Pi-Zero)

Physical Intelligence, 2024 (open weights released later via their `openpi` repository). Pushes further than discretized tokens (RT-2/OpenVLA) or per-chunk diffusion (Octo): it uses **flow matching** to generate continuous, high-frequency action trajectories directly, aimed at more dexterous tasks (e.g. folding laundry, plugging in a cable) that need smoother, faster control than token-by-token generation allows. Built on a VLM backbone (PaliGemma) with an added "action expert" module.

3.5 Generalist Robot Policies (Concept)

The category RT-2, OpenVLA, Octo, and π_0 all belong to: a single trained model that transfers across tasks, objects, scenes, and sometimes even different robot bodies (cross-embodiment), as opposed to the older paradigm of training one narrow policy per task from scratch.

3.6 Vision-Language-Conditioned Control (Concept)

The mechanism by which a model architecturally fuses a camera image and a language instruction into a shared representation that then conditions the action-generation process. This is the technical thread that differentiates the four models: discrete tokens with autoregressive decoding (RT-2/OpenVLA), a diffusion action head (Octo), and flow matching (π_0) – three different answers to the same underlying question of how to turn (image, language) into a continuous robot action.

4 What the Application / Demo Looks Like in Practice

Regardless of which model is used, a working VLA system always runs the same control loop:

1. A camera captures an RGB frame of the workspace.
2. A natural-language command is provided, e.g. *"pick up the red block and place it on the plate."*
3. Model forward pass: (image, instruction) $\rightarrow$ action.
4. The action – either a single 7-DoF vector, or a short chunk of future actions for Octo/π_0 – is sent to the robot's controller, typically as a target end-effector delta-pose plus a gripper command.
5. The loop repeats at some control frequency (a few Hz for token-based models like OpenVLA; higher for chunked/continuous models like Octo/π_0) until the task is judged complete.

For OpenVLA and Octo specifically, this is exactly what their public example/inference code does out of the box: load the model, pass in an image and an instruction, and get an action vector back. Whether that action is then executed on a real robot arm, a simulated arm, or simply visualized/printed is an implementation decision independent of the model itself, and can be adapted to whatever hardware or simulation environment is available for the demo (real robot, simulation, or visualization-only).

5 Demo Planning Note

The presentation covers all six subtopics above. The live demo component may focus on a subset of the four concrete models (OpenVLA and Octo, or up to three including π_0), depending on what hardware/simulation access is confirmed and how reliably each model's public inference code can be set up in advance of the session.